

Mobile Application Development

**Networking in Jetpack Compose
using Retrofit, Coroutines, StateFlow,
and Coil for image loading**



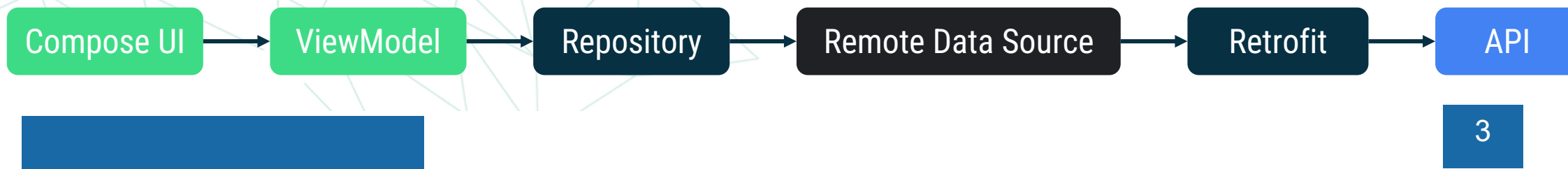
Connect to the Internet

Using Retrofit for Network Calls

- **Retrofit: A type-safe HTTP client** for Android and Java/Kotlin (built on top of OkHTTP).
- Simplifies RESTful API integration (REST API calls and JSON conversion).
- Works smoothly with **Kotlin coroutines** and **data classes**.

Why Retrofit?

- Easy API definition.
- Compatible with JSON parsing libraries (Gson/Moshi).
- Supports asynchronous calls with coroutines.



Defining API Service (Retrofit Interface)

- Define endpoints as Kotlin interface methods:

```
interface ApiService {  
    @GET("users")  
    suspend fun getUsers(): List<User>  
}
```

Data Models with Gson/Moshi

- Map JSON responses to Kotlin data classes:

Example using Gson/Moshi:

```
data class User(  
    val id: Int,  
    val name: String,  
    val email: String  
)
```

- Gson/Moshi automatically converts JSON ↔ Kotlin objects.

Asynchronous API Calls with Suspend Functions

- Retrofit supports Kotlin Coroutines for async network calls.

Example API Call:

```
val users = apiService.getUsers() // Suspend function
```

- Coroutine suspends and resumes without blocking the main thread.

Using Coroutines in Compose

- Compose uses Coroutines effectively for async tasks.

Important Coroutine scopes:

- **LaunchedEffect**: **Lifecycle-aware coroutine scope** tied to a composable.
- **rememberCoroutineScope**: For a coroutine triggered by UI events (e.g., clicks).

LaunchedEffect Example

- **LaunchedEffect**: Triggers on composition, recomposes when key changes.

```
LaunchedEffect(Unit) {  
    val data = apiService.getUsers()  
    usersState.value = data  
}
```


LaunchedEffect

What You Want	Example	Key to Use
Run once when screen opens	<code>LaunchedEffect (Unit)</code>	<code>Unit</code>
Re-run when a parameter changes	<code>LaunchedEffect (userId)</code>	<code>variable</code>
Re-run when multiple things change	<code>LaunchedEffect (userId, filter)</code>	<code>multiple</code>
Never restart automatically	Don't use <code>LaunchedEffect</code>	Use <code>rememberCoroutineScope</code>

rememberCoroutineScope Example

- **rememberCoroutineScope**: For user interaction events.

```
val scope = rememberCoroutineScope()

Button(onClick = {
    scope.launch {
        val users = apiService.getUsers()
    }
}) {
    Text("Fetch Users")
}
```

Displaying Data from the Network

Typical pattern:

- Fetch data in ViewModel.
- Expose state to UI via StateFlow.
- Display in LazyColumn.

Example:

```
LazyColumn {  
    items(usersList) { user ->  
        Text("${user.name} (${user.email})")  
    }  
}
```

Loading Indicators and Error States

Show loading and errors gracefully:

```
when (uiState) {  
    is UiState.Loading -> CircularProgressIndicator()  
    is UiState.Success -> UserList((uiState as UiState.Success).data)  
    is UiState.Error -> Text("Error fetching data")  
}
```

- Provides smooth UX feedback.

Best Practices for Networking

- **ViewModel** handles all network logic, exposing data as observable state.
- **UI** observes changes through reactive streams (**StateFlow** or **LiveData**).
- Separate UI logic from networking clearly.

ViewModel Handling API & State

ViewModel example:

```
class UserViewModel : ViewModel() {  
    private val _users = MutableStateFlow<UiState<List<User>>>(UiState.Loading)  
    val users = _users.asStateFlow()  
  
    init {  
        fetchUsers()  
    }  
  
    private fun fetchUsers() {  
        viewModelScope.launch {  
            try {  
                val result = apiService.getUsers()  
                _users.value = UiState.Success(result)  
            } catch (e: Exception) {  
                _users.value = UiState.Error(e.message)  
            }  
        }  
    }  
}
```

UI Observing State

Compose UI observes the ViewModel state reactively:

```
@Composable
```

```
fun UserScreen(viewModel: UserViewModel) {
```

```
    val uiState by viewModel.users.collectAsState()
```

```
    when (uiState) {
```

```
        is UiState.Loading -> CircularProgressIndicator()
```

```
        is UiState.Success -> UserList((uiState as UiState.Success).data)
```

```
        is UiState.Error -> ErrorMessage((uiState as UiState.Error).errorMessage)
```

```
    }
```

```
}
```

Handling Network Errors

- Show informative UI (snackbars, dialogs).
- Implement retry mechanisms.
- Handle offline gracefully.

Snackbar/Error Example:

```
if (uiState is UiState.Error) {  
    Snackbar(  
        action = {  
            Button(onClick = { viewModel.retryFetch() }) { Text("Retry") }  
        }  
    ) {  
        Text("Failed to load data.")  
    }  
}
```


Retry Logic & Offline States

Example Retry Function in ViewModel:

```
fun retryFetch() {  
    fetchUsers()  
}
```

- Consider offline caching to enhance user experience:
 - Cache the last successful network response.
 - Display cached data when offline.

Image Loading in Compose

- Use **Coil** library for easy image loading.

Why Coil?

- Modern image loading library optimized for Jetpack Compose.
- Automatically image caching, handles placeholders, and errors.

Using Coil in Compose

Basic usage:

```
AsyncImage(  
    model = "https://example.com/user.png",  
    contentDescription = "User Image",  
    placeholder = painterResource(R.drawable.placeholder),  
    error = painterResource(R.drawable.error)  
)
```

- Simple, declarative, and efficient.

Placeholders and Error Images

- Improve UX during loading/errors:

```
AsyncImage(  
    model = imageUrl,  
    contentDescription = "Profile picture",  
    placeholder = painterResource(R.drawable.loading),  
    error = painterResource(R.drawable.error_image)  
)
```

Quiz Time

Key Takeaways

- Retrofit simplifies network calls.
- Use suspend functions with coroutines for efficient network operations.
- ViewModels manage network state; Compose UI observes reactively.
- Handle loading, errors, and retry mechanisms gracefully.
- Coil simplifies efficient image loading with Compose.

THANK YOU